Смотри мне кажется с агентам, координатором, и роями надо еще поработать попробуй ещё нарастить, подумай что упущено и интегрируй на лету, может быть работа с узлами? Как предложение

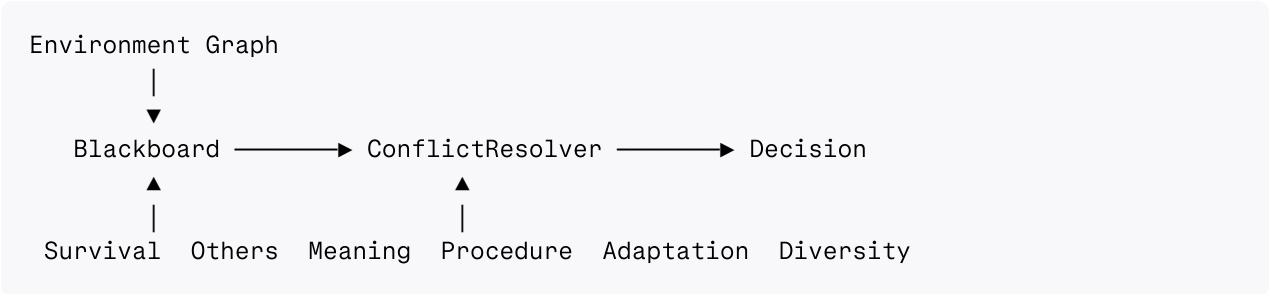
Да. Сейчас у вас есть **агенты и координатор**, но не хватает трёх вещей: **узлов-среды**, **графа связей** и **общей доски состояния**. Тогда рой перестанет быть набором шести независимых оценщиков и станет системой, где агенты влияют друг на друга через структуру, а не через «средний голос».

Что упущено

Компонент	Зачем нужен
`Node`	Узел среды: студент, группа, факультет, экзамен, правило
`Edge`	Связь и влияние: «студент входит в группу», «правило ограничивает преподавателя»
`Blackboard`	Общая доска: факты, решения, возражения, история
`GraphSwarm`	Рой на графе: агент видит не всё, а свой локальный контекст
`ConflictResolver`	Не усредняет мнения, а классифицирует конфликт и эскалирует
`Provenance`	Происхождение решения: кто, когда, на каком основании

Граф здесь особенно важен: узлы представляют агентов или объекты среды, а рёбра задают зависимости и поток информации; общее состояние читается и обновляется узлами по мере выполнения. [1] [2] [3]

Архитектура



Ключевой сдвиг: **координатор больше не главный судья.** Он лишь читает граф, запускает агентов, собирает их сообщения в blackboard и применяет правила разрешения

конфликтов. Это ближе к blackboard-архитектуре, где агенты читают и пишут в общее структурированное пространство, а контроллер активирует нужных участников. [4] [5] [6]

1. `src/les/swarm/graph.py`

```
from dataclasses import dataclass, field
from typing import Dict, List, Set, Tuple


@dataclass
class Node:
    node_id: str
    node_type: str
    attributes: Dict[str, float] = field(default_factory=dict)


@dataclass
class Edge:
    source: str
    target: str
    relation: str
    weight: float = 1.0


class EnvironmentGraph:
    def __init__(self):
        self.nodes: Dict[str, Node] = {}
        self.edges: List[Edge] = []
        self.adjacency: Dict[str, Set[str]] = {}

    def add_node(self, node: Node) -> None:
        self.nodes[node.node_id] = node
        self.adjacency.setdefault(node.node_id, set())

    def add_edge(self, edge: Edge) -> None:
        if edge.source not in self.nodes or edge.target not in self.nodes:
            raise ValueError("Edge references unknown node")

        self.edges.append(edge)
        self.adjacency[edge.source].add(edge.target)

    def neighbors(self, node_id: str) -> Set[str]:
        return self.adjacency.get(node_id, set())

    def local_state(self, node_id: str) -> Dict[str, float]:
        node = self.nodes[node_id]
        state = dict(node.attributes)

        for neighbor_id in self.neighbors(node_id):
            neighbor = self.nodes[neighbor_id]
            for key, value in neighbor.attributes.items():
                state[f"neighbor_{neighbor_id}_{key}"] = value

        return state
```

2. `src/les/swarm/blackboard.py`

```
import time
import uuid
from typing import Any, Dict, List


class Blackboard:
    """
    Общая доска состояния.

    Хранит не «среднее мнение», а отдельные записи:
    факт, интерпретацию, возражение, решение.
    """

    def __init__(self):
        self.entries: List[Dict[str, Any]] = []

    def write(
        self,
        author: str,
        entry_type: str,
        payload: Dict[str, Any],
        provenance: Dict[str, Any] | None = None,
    ) -> str:
        entry_id = str(uuid.uuid4())

        self.entries.append({
            "id": entry_id,
            "timestamp": time.time(),
            "author": author,
            "type": entry_type,
            "payload": payload,
            "provenance": provenance or {},
        })

        return entry_id

    def read(self, entry_type: str | None = None) -> List[dict]:
        if entry_type is None:
            return self.entries.copy()

        return [
            entry for entry in self.entries
            if entry["type"] == entry_type
        ]

    def conflicts(self) -> List[dict]:
        return self.read("conflict")
```

3. `src/les/swarm/conflict_resolver.py`

```
from typing import Dict, List


class ConflictResolver:
    """
    Не усредняет решения.
    Определяет тип конфликта и выбирает процедуру.
    """

    def resolve(self, messages: List[dict]) -> Dict[str, object]:
        blocks = [m for m in messages if m.get("action") == "block"]
        escalations = [m for m in messages if m.get("action") == "escalate"]
        errors = [m for m in messages if m.get("action") == "error"]

        if blocks:
            return {
                "decision": "STOP",
                "reason": "Обнаружен блокирующий конфликт",
                "involved": [m["agent"] for m in blocks],
            }

        if errors:
            return {
                "decision": "ESCALATE",
                "reason": "Ошибка агента требует проверки",
                "involved": [m["agent"] for m in errors],
            }

        if escalations:
            return {
                "decision": "ESCALATE",
                "reason": "Агенты сообщают о нарушении порога",
                "involved": [m["agent"] for m in escalations],
            }

        return {
            "decision": "CONTINUE",
            "reason": "Конфликтов не обнаружено",
            "involved": [],
        }
```

4. `src/les/swarm/graph_swarm.py`

```
import asyncio
import logging
from typing import Dict, List, Any

from les.agents import BaseAgent
from .graph import EnvironmentGraph
from .blackboard import Blackboard
from .conflict_resolver import ConflictResolver
```

```
logger = logging.getLogger(__name__)


class GraphSwarm:
    def __init__(
        self,
        agents: List[BaseAgent],
        graph: EnvironmentGraph,
    ):
        self.agents = agents
        self.graph = graph
        self.blackboard = Blackboard()
        self.resolver = ConflictResolver()

    async def run(self, focus_node_id: str) -> Dict[str, Any]:
        if focus_node_id not in self.graph.nodes:
            raise ValueError(f"Unknown node: {focus_node_id}")

        local_state = self.graph.local_state(focus_node_id)

        tasks = [
            agent.safe_act(local_state)
            for agent in self.agents
        ]

        raw_results = await asyncio.gather(
            *tasks,
            return_exceptions=True,
        )

        messages: List[dict] = []
        errors: List[dict] = []

        for agent, result in zip(self.agents, raw_results):
            if isinstance(result, BaseException):
                errors.append({
                    "agent": agent.name,
                    "error": str(result),
                })
                continue

            messages.append(result)

            self.blackboard.write(
                author=agent.name,
                entry_type="assessment",
                payload=result,
                provenance={
                    "focus_node": focus_node_id,
                    "local_state": local_state,
                },
            )

        if errors:
            self.blackboard.write(
                author="system",
```

```
                entry_type="error",
                payload={"errors": errors},
            )

        decision = self.resolver.resolve(messages)

        self.blackboard.write(
            author="conflict_resolver",
            entry_type="decision",
            payload=decision,
        )

        return {
            "focus_node": focus_node_id,
            "local_state": local_state,
            "messages": messages,
            "errors": errors,
            "decision": decision,
        }
```

5. main.py

```
import asyncio
from les.agents import (
    SurvivalAgent,
    OthersAgent,
    MeaningAgent,
    ProcedureAgent,
    AdaptationAgent,
    DiversityAgent,
)
from les.swarm.graph import Node, Edge, EnvironmentGraph
from les.swarm.graph_swarm import GraphSwarm


async def main():
    graph = EnvironmentGraph()

    graph.add_node(Node(
        node_id="student_1",
        node_type="student",
        attributes={
            "S": 0.8,
            "O": 0.8,
            "M": 0.8,
            "P": 0.8,
            "A": 0.8,
            "D": 0.2,
        },
    ))

    graph.add_node(Node(
        node_id="group_A",
        node_type="group",
        attributes={
```

```
            "average_pressure": 0.9,
            "diversity": 0.2,
        },
    ))

    graph.add_node(Node(
        node_id="exam_rule",
        node_type="rule",
        attributes={
            "individual_criteria": 1.0,
            "group_average_required": 0.0,
        },
    ))

    graph.add_edge(Edge(
        source="student_1",
        target="group_A",
        relation="belongs_to",
    ))

    graph.add_edge(Edge(
        source="exam_rule",
        target="student_1",
        relation="governs",
    ))

    agents = [
        SurvivalAgent("survival"),
        OthersAgent("others"),
        MeaningAgent("meaning"),
        ProcedureAgent("procedure"),
        AdaptationAgent("adaptation"),
        DiversityAgent("diversity"),
    ]

    swarm = GraphSwarm(agents, graph)

    result = await swarm.run("student_1")

    print("Фокусный узел:", result["focus_node"])
    print("Решение:", result["decision"])
    print("Сообщения агентов:", result["messages"])
    print("Ошибки:", result["errors"])
    print("Записей на доске:", len(swarm.blackboard.entries))


if __name__ == "__main__":
    asyncio.run(main())
```

Что это меняет

Теперь рой работает не с абстрактным вектором из шести чисел, а с **контекстом узла**:

- `student_1` видит свою группу;
- `group_A` несёт давление усреднения;
- `exam_rule` задаёт процедурную границу;
- `DiversityAgent` видит, что различие подавлено;
- `ConflictResolver` не усредняет, а фиксирует блокирующий конфликт.

Это уже не «шесть агентов голосуют», а **графовая система**: узлы делают работу, рёбра определяют зависимости и передачу информации, а общее состояние связывает участников. [3] [7] [1]

Следующий уровень

Следующие расширения, которые я бы добавил:

- `Memory`: локальная память агента — что он уже видел и какие решения принимал.
- `Reputation`: вес агента зависит не от статуса, а от точности прошлых прогнозов.
- `AppealNode`: отдельный узел апелляции, доступный студенту.
- `AuditTrail`: неизменяемый журнал: кто, когда и на каком основании изменил оценку.
- `PolicyEngine`: правила в YAML, а не в коде — чтобы критерии можно было менять без переписывания агентов.

Главная формула теперь такая:

> **Не «все согласны», а «каждый виден, каждый различим, каждый может возразить».**

Это и есть содержательный ответ на усреднение: не сделать всех одинаковыми, а сделать различие **видимым, защищённым и процедурно значимым**. [5] [8] [4]

⁂

1. https://strandsagents.com/latest/documentation/docs/user-guide/concepts/multi-agent/graph/
2. https://ppe-simulations.pacuit.org/python-basics/networks
3. https://www.datacamp.com/tutorial/graph-engineering-guide
4. https://ar5iv.labs.arxiv.org/html/2507.01701
5. https://www.autolearningagents.com/multi-agent-systems/shared-memory.php
6. https://github.com/roanbrasil/agents-integration-patterns/blob/main/patterns/messaging/blackboard.md
7. https://psssnikhil.github.io/ai-engineering-hub/build/module-12-multi-agent-systems/lessons/08-shared-memory-and-blackboards/
8. https://people.acciona.com/organizational-culture/groupthink
9. https://arxiv.org/html/2610.14515v1

10. https://arxiv.org/html/2605.29313v1
11. https://www.premai.io/blog/multi-agent-ai-systems-architecture-communication-and-coordination/
12. https://www.cognee.ai/stateful-multi-agent-systems
13. https://agent-atlas.blackarcindustrial.com/papers/graph-engineering-anthropic-playbook/
14. https://github.com/microsoft/UFO/blob/main/documents/docs/infrastructure/agents/design/memory.md
15. https://github.com/mariopaolucci/agent-based-evolutionary-game-dynamics-in-python/blob/main/nxn-Imitate-If-Better-Networks.py
16. http://www.link.springer.com/article/10.1007/s11403-025-00462-2